Aéza Analyzer

English version of ../README.md.

A web application for analysing service renewal risk and revenue at risk. Stack: FastAPI, React, PostgreSQL and Docker Compose.

Running

Docker and Docker Compose are required. Create `.env`, set your own database password and start the application:

```
Copy-Item .env.example .env
# Set POSTGRES_PASSWORD in .env
docker compose up --build -d
docker compose exec api python -m backend.users create analyst@example.com
```

UI: http://localhost:8080. The API and its reference: http://localhost:8000/api/v1/docs.

Data and scores

On the first start the application loads `dataset/data.json`: a synthetic set of 10 accounts, 30 services and 100 transactions. It can be replaced through `PUT /api/v1/source/snapshot`.

The risk score is an explainable weighted heuristic, not a churn probability: the set contains no labelled outcomes. Revenue at risk is computed separately from the price of the nearest renewal and the risk score of the service.

Verification

```
make doctor          # the environment: tools, .env, ports (changes nothing)
make test            # backend + the authentication module
make openapi-check   # the API specification matches the application
make docs-check      # documentation, links, module specifications
make verify          # everything above at once
```

Without make: `python -m pytest backend/tests -q`, `python -m pytest tests/test_auth.py -q`, `cd frontend && npm ci && npm run build`.

The full "claim → command → CI" matrix: `docs/verification/matrix.md`.

Documentation

Every document exists in Russian and English and in two formats — `.md` and `.pdf` (`docs/pdf/ru/`, `docs/pdf/en/`); a language divergence is caught by `make docs-check`. The module map for borrowing code: `docs/modules/module-map.md`.

The entry point is **docs/index.md** (the map of all documents).

- Building from a clean machine — pinned versions, levels A–E
- Running and upgrading · Design and scoring

- Schema and migrations · API
- Module specifications · Operations
- Verification matrix · Portability audit

License

MIT — see LICENSE. Use, modification and distribution are permitted provided the copyright notice is retained.

Project management: `forge.sh` and `forge.bat`

One script for Unix-like systems (`forge.sh`) and one for Windows (`forge.bat`). The interface is identical: the same commands, the same logic, one implementation per platform — no bash scripts inside Windows.

```
./forge.sh              # interactive menu
./forge.sh doctor       # check the environment, changing nothing
./forge.sh install      # install the dependencies
./forge.sh start        # start in the background, wait for readiness, open the browser
./forge.sh status       # what is running and whether the port answers
./forge.sh logs api -f  # logs in real time
./forge.sh test         # tests
./forge.sh stop         # stop
```

```
forge.bat
forge.bat start
forge.bat status
```

What it can do

Command	Action
`status`	state by ports and HTTP, not "by eye"
`start [--fg]`	background (PID files in `.run/`) or foreground
`stop`	stops the whole process tree, the port is released
`restart`, `logs [name] [-f]`, `open`	the usual operations
`doctor`	tools, versions, `.env`, `.env` in git, port occupancy
`install`, `test`, `lint`, `build`, `verify`	through `make`, if it exists
`db migrate|backup|restore`	the database: migrations, backup, restore
`docker up|down|logs|reset`	containers (`down` does not delete data)
`clean [--hard]`	artefacts and logs; `--hard` also `.venv` and `node_modules`

Rules built into the script

- **It changes nothing without a command.** `doctor` and `status` only read.
- **Data is never deleted.** `stop` and `docker down` keep the database; deletion is only `--hard` or an explicit confirmation in `docker reset`.

- **Pre-flight checks.** If `go`/python is missing or a port is taken, the script says so immediately, not after a two-minute timeout.
- **It does not hang after a crash.** If the process dies, start-up stops and shows the last lines of the log.
- **The venv wins over the system.** If `.venv` exists, `make`, `pytest` and uvicorn use it — so that `install` and `test` do not diverge.

The current way to run without the script: `uvicorn main:app --port 8000` — the address is `http://localhost:8000/api/v1/docs`.

Configuration: `.env`

The script reads `.env` **with its own parser**, not through `source`: the file comes from the repository, and arbitrary code must not be executed from it. Supported are `#` comments, empty lines, the `export` prefix, single and double quotes, CRLF, spaces around `=` and a comment after a value.

Value priority: **environment variable → `.env` → built-in value**. So `PORT=9000 ./forge.sh start` works regardless of the file.

Command	Action
`env` / `env show`	the variables, the source of each value and masked secrets
`env init [--force]`	create `.env` from `.env.example`: secret placeholders are replaced with random values, permissions `600`, the previous file goes to `.env.bak`
`env set KEY=VALUE`	write or replace a single variable (atomically, through a temporary file)
`env check`	compare with `.env.example`: missing variables, empty and "sample" secrets, `.env` in `.gitignore`
`env edit`	open the file in `$EDITOR` (fallback: nano → vim → vi)
`env path`, `env reload`	the path to the file and a re-read without a restart

Secrets are masked in the output: `ADMIN_CODE=ch…de (31 chars)`, the password inside a connection string — `postgres://user:***@host`. Names like `*_MINUTES`, `*_URL`, `*_PORT` are not treated as secrets: masking a number of minutes would be pointless.

The path to the file can be overridden: `FORGE_ENV_FILE=/etc/app.env ./forge.sh status`. The Windows version has the same commands: `forge.bat env init`, `forge.bat env check`. A `cmd.exe` limitation: values containing `%` and `!` are not supported.

Flags and completion

```
./forge.sh -h | --help           # help
./forge.sh -V | --version        # the script version, the commit, the state of .env
./forge.sh -q status             # say nothing about successes
./forge.sh --no-color status     # no ANSI (the same as NO_COLOR=1)
./forge.sh --json status         # machine output for scripts and monitoring

eval "$(./forge.sh completion bash)"   # bash completion
./forge.sh completion zsh > ~/.zfunc/_forge   # for zsh
```

`test`, `lint`, `build` and `verify` run in an environment **without** the variables from `.env`: otherwise local checks would behave differently from CI, and tests like "without a key there must be a fallback" would stop verifying anything.

Running in this project

```
./forge.sh install && ./forge.sh start
```